

Como usar este material

- Este documento é uma adaptação da atividade original do professor (migração de um projeto HTML/Tailwind para Next.js), trocando a temática de e-commerce por um painel de ferramentas de cibersegurança. A diferença principal em relação à versão que o ChatGPT te enviou:
- **Toda a estilização em Tailwind já vem pronta** nos códigos de partida — você não precisa escrever nenhuma classe do zero.
 - As tarefas marcadas como **TODO ALUNA** são **exclusivamente de Next.js, React e TypeScript**: tipagem de props, estados, useEffect, fetch, .map(), next/image.
 - A Fase 0 (HTML puro) é opcional — ela existe só para você comparar "antes e depois", não precisa recriá-la.

Contexto

Você foi responsável por modernizar um pequeno painel web chamado **CyberLab**, que apresenta ferramentas de segurança da informação reais (Nmap, Wireshark, Metasploit, OWASP ZAP, entre outras), consumidas em tempo real através da API pública do GitHub.

A versão legada foi feita em HTML5 + Tailwind CSS via CDN. Sua tarefa é migrar essa interface para Next.js com TypeScript, transformando a página estática em uma aplicação dinâmica com um contador de "Toolkit" (ferramentas selecionadas).

Objetivos de aprendizagem

Você vai praticar	Você NÃO vai praticar aqui
Criar um projeto Next.js (App Router) TypeScript e interfaces Componentes reutilizáveis e props tipadas useState e useEffect Consumo de API com fetch Renderização com .map() next/image e next/font next.config.ts (domínios de imagem externa)	Escrever classes Tailwind do zero Criar layout/grid/responsividade do zero

Estrutura esperada do projeto

```
cyberlab-next/  
├── public/  
├── src/  
│   ├── app/  
│   │   ├── globals.css  
│   │   ├── layout.tsx      <-- pronto (só falta trocar a fonte)  
│   │   └── page.tsx        <-- visual pronto, lógica é sua  
│   └── components/  
│       └── CardFerramenta.tsx <-- visual pronto, lógica é sua  
├── next.config.ts          <-- pronto  
├── package.json  
├── tsconfig.json  
└── README.md
```

Fase 0 (opcional) — Projeto legado, só para referência

O painel original em HTML puro tinha um cabeçalho "CyberLab / Security Tools & Resources", um contador "Ferramentas selecionadas: 0" e cards com imagem, nome, categoria, descrição e um botão "Adicionar ao Toolkit". Não é necessário recriar esse arquivo — é só o "antes" da migração, para você entender o que está sendo transformado em componentes dinâmicos.

Fase 1 — Criar o projeto Next.js

No terminal:

```
npx create-next-app@latest cyberlab-next
```

Selecione:

Opção	Resposta
TypeScript	Yes

ESLint	Yes
Tailwind CSS	Yes
src/ directory	Yes
App Router	Yes

```
cd cyberlab-next
code . -r
```

Fase 2 — Configuração pronta: next.config.ts

Já pronto. Só copie para `next.config.ts` , na raiz do projeto.

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "avatars.githubusercontent.com",
      },
    ],
  },
};

export default nextConfig;
```

Fase 3 — src/app/layout.tsx

1 TODO só de Next.js: trocar a fonte Inter pela Montserrat (é um requisito explícito do professor no documento original, e é sobre next/font, não sobre Tailwind).

```
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";

// TODO ALUNA: importar e usar a fonte Montserrat do next/font/google no lugar da Inter
const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "CyberLab - Security Tools & Resources",
  description: "Atividade prática de Next.js, API REST e TypeScript",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="pt-BR">
      <body className={`${inter.className} bg-slate-900 text-white min-h-screen`}>
        {children}
      </body>
    </html>
  );
}
```

Fase 4 — src/components/CardFerramenta.tsx

Visual 100% pronto em Tailwind. Você só preenche a parte de TypeScript/React.

```
import Image from "next/image";

// TODO ALUNA – PASSO A: criar a interface 'FerramentaProps' contendo:
// id: number; nome: string; descricao: string; imagem: string;
// url: string; estrelas: number; onAdicionar: () => void;

export default function CardFerramenta(/* TODO ALUNA – PASSO B: receber as props tipadas aqui, usando a interface acima */) {
  return (
    <div className="bg-slate-800 border border-slate-700 rounded-xl overflow-hidden shadow-md flex flex-col justify-between hover:border-emerald-500 transition-all duration-300">
      <div className="bg-white p-4 flex items-center justify-center h-32 relative">
        {/* TODO ALUNA: trocar o src fixo pela prop 'imagem' */}
        <Image
          src="https://avatars.githubusercontent.com/u/512584?v=4"
          alt="Logo da ferramenta"
          fill
          className="object-contain p-2"
        />
      </div>
    </div>
  );
}
```

```

<div className="p-5 flex flex-col flex-1 justify-between space-y-4">
  <div>
    <span className="text-xs uppercase font-semibold text-emerald-400 tracking-wider">
      {/* TODO ALUNA: exibir a prop 'estrelas', ex: * {estrelas} */}
      * 0
    </span>
    <h2 className="font-bold text-white text-lg line-clamp-2 mt-1">
      {/* TODO ALUNA: exibir a prop 'nome' */}
      Nome da ferramenta
    </h2>
    <p className="text-slate-400 text-sm line-clamp-2 mt-2">
      {/* TODO ALUNA: exibir a prop 'descricao' */}
      Descrição da ferramenta retornada pela API.
    </p>
  </div>
  <div className="pt-4 border-t border-slate-700 flex items-center justify-between">
    <a
      /* TODO ALUNA: usar a prop 'url' no href */
      href="#"
      target=" blank"
      className="text-xs text-slate-400 underline hover:text-emerald-400"
    >
      Ver no GitHub
    </a>
    <button
      onClick={() => {
        /* TODO ALUNA: chamar a função recebida via prop 'onAdicionar' */
      }}
      className="bg-emerald-600 hover:bg-emerald-500 text-white text-sm font-medium px-3 py-2 rounded-lg transition"
    >
      Adicionar ao Toolkit
    </button>
  </div>
</div>
</div>
</div>
);
}

```

Fase 5 — API: GitHub Search (repositórios de cibersegurança)

Em vez da FakeStoreAPI, você vai consumir repositórios reais de ferramentas de segurança no GitHub:

```
https://api.github.com/search/repositories?q=topic:cybersecurity&sort=stars&order=desc&per_page=6
```

Atenção: essa API retorna os repositórios dentro de um campo chamado `items` (é um array dentro de um objeto, diferente da FakeStoreAPI que devolve o array direto). É um bom exercício real de leitura de resposta de API. Cada item tem, entre outros: `id`, `name`, `description`, `html_url`, `stargazers_count`, `owner.avatar_url`.

Limite de requisições: a API do GitHub sem autenticação permite 60 requisições por hora por IP — mais do que suficiente para estudar, mas evite dar refresh sem parar na página.

Fase 6 — src/app/page.tsx

Todo o JSX/Tailwind do cabeçalho e do grid já está pronto abaixo. Os PASSOS são só lógica React/TypeScript.

```

"use client";

// PASSO 1: importar useState e useEffect do React
// PASSO 1A: importar o componente CardFerramenta do caminho correto

// PASSO 2: criar a interface 'Ferramenta' com os campos que você vai
// usar no componente: id, nome, descricao, imagem, url, estrelas
// (você decide como mapear os nomes da API do GitHub para esses campos)

export default function Home() {
  // PASSO 3: criar o estado 'ferramentas' (array de Ferramenta, iniciar em [])
  // PASSO 4: criar o estado 'toolkit' (number, iniciar em 0)
  // PASSO 5: criar o estado 'loading' (boolean, iniciar em true)

  // PASSO 6: usar o useEffect para buscar os dados na API ao carregar a página
  // Endereço: https://api.github.com/search/repositories?q=topic:cybersecurity&sort=stars&order=desc&per_page=6
  // Lembrete: a lista de repositórios vem dentro de data.items
  useEffect(() => {
    // fetch("https://api.github.com/search/repositories?q=topic:cybersecurity&sort=stars&order=desc&per_page=6")
    // .then(res => res.json())
    // .then(data => { guardar data.items (já mapeado para 'Ferramenta') no estado 'ferramentas' e mudar 'loading' para false })
  }, []);

  const adicionarAoToolkit = () => {
    // PASSO 7: incrementar o estado 'toolkit' em +1
  };

  return (
    <main className="max-w-6xl mx-auto p-8">
      <header className="mb-8 flex justify-between items-center border-b border-slate-800 pb-4">
        <div>
          <h1 className="text-3xl font-bold text-emerald-400">CyberLab</h1>
          <p className="text-slate-400 text-sm">Security Tools & Resources</p>
        </div>
        <div className="bg-slate-800 px-4 py-2 rounded-lg border border-slate-700">

```

```
        {/* PASSO 8: trocar o '0' fixo pela variável de estado 'toolkit' */}
        Toolkit: <span className="font-bold text-emerald-400">0</span>
      </div>
    </header>

    {/* TODO ALUNA: exibir "Carregando ferramentas..." quando 'loading' for true */}

    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
      {/* PASSO 9: usar .map() em 'ferramentas' para renderizar <CardFerramenta /> */}
      {/* PASSO 10: passar as props corretas, incluindo onAdicionar={adicionarAoToolkit} */}
    </div>
  </main>
);
}
```

Fase 7 — README.md

```
# CyberLab

## Desenvolvedora
Isadora Umlauf

## Objetivo
Migrar uma aplicação HTML + Tailwind CSS para Next.js utilizando TypeScript.

## Tecnologias
- Next.js
- TypeScript
- React
- Tailwind CSS
- REST API (GitHub Search API)
- next/image
- next/font

## Funcionalidades
- Consumo de API em tempo real
- Listagem dinâmica de ferramentas de cibersegurança
- Componente reutilizável (CardFerramenta)
- Contador do Toolkit
- Estado de carregamento
- Renderização utilizando map()

## Desenvolvimento
Atividade desenvolvida individualmente, adaptando o projeto original
de e-commerce para uma aplicação de ferramentas de cibersegurança.
```

Checklist de estudo para a prova

Use isto para treinar os conceitos, sem depender do código pronto:

Next.js	React
☐ Criar projeto com create-next-app ☐ Entender src/app, page.tsx, layout.tsx ☐ Entender App Router ☐ next/image e configuração de domínios externos ☐ next/font/google	☐ O que é um componente ☐ O que são props e como tipá-las ☐ useState: quando e como usar ☐ useEffect: quando e por quê ☐ Como atualizar um estado ☐ Como usar .map() para renderizar listas
TypeScript	API / fetch
☐ Criar uma interface ☐ Tipar props e arrays ☐ string, number, boolean ☐ Tipar uma função: () => void	☐ O que é uma API REST ☐ O que faz fetch() ☐ O que é response.json() ☐ Guardar dados da API no estado ☐ Por que usar useEffect para requisições ☐ Ler um array aninhado (ex: data.items)

Dica final: como o Tailwind já vem pronto em todos os arquivos, se sobrar tempo de estudo, o melhor uso dele é: (1) recriar o CardFerramenta.tsx e o page.tsx do zero, olhando só o enunciado, sem colar o código pronto; e (2) tentar trocar a API do GitHub por outra API pública qualquer, só para praticar fetch + useEffect + interface em um contexto diferente.